ARCHITECTURE DOCUMENT

Grid-Guard AI

System Architecture

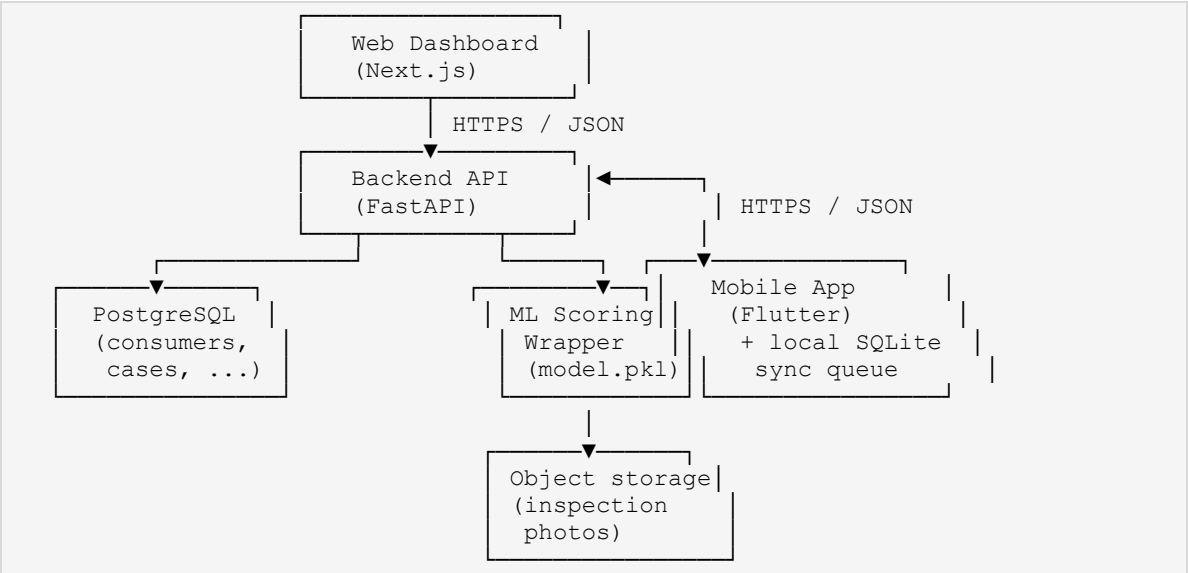
Component responsibilities, data flow, deployment topology, and security architecture

Field	Value
Author	Noor Eman
Version	1.0 (Draft)
Related documents	Technical Design Document; API Specification; ML Data, Model & Validation Strategy

1. Architectural Style

Grid-Guard AI is a thin-client architecture: the web dashboard and mobile app contain no business logic of their own beyond local caching and offline-queueing. All scoring, state transitions, and authorization live in a single backend service. This keeps the two client codebases simple and ensures both surfaces always see consistent case/consumer state — there is exactly one source of truth.

2. Component Diagram (Textual)



3. Component Responsibilities

Component	Responsibility	Does NOT do
Web Dashboard	Render officer-facing views; call backend API; client-side filtering/sorting of already-fetched data	Never computes risk scores or writes to the DB directly — always via API
Mobile App	Render inspector-facing views; cache assigned cases locally; queue inspection submissions offline; sync when connected	Never contains its own copy of business rules (e.g. what counts as "confirmed") — the server is authoritative
Backend API	Auth, authorization, all state transitions (assign/close case), calls the ML scoring wrapper, writes audit log entries for every mutation	Never trusts client-supplied risk scores or explanations — always recomputed/read from the risk_scores table
ML Scoring Wrapper	Loads the trained model once at startup; exposes a single score(consumer_features) → fixed-shape output function (per the ML Output Schema in the Technical Design Document)	Never retrains itself automatically in v1 — retraining is a manual, offline process using the ML pipeline notebooks
PostgreSQL	Single source of truth for all entities	—

Component	Responsibility	Does NOT do
Object storage	Stores inspection photos; returns a URL referenced from the inspections table	Never stores structured data — photos only

4. Data Flow — Flag to Feedback (End-to-End)

This is the same loop walked through narratively in the user stories; here it is stated as a numbered technical sequence.

1. Offline ML pipeline (per the ML Strategy document) produces model.pkl; this is deployed to the backend's ml_artifacts/ directory.
2. A scoring run (manually triggered for v1, since there is no live AMI feed to trigger it automatically) computes risk_score rows for all consumers and writes them to the database, plus an audit_log entry (flag_generated) per newly high-risk consumer.
3. Web Dashboard calls GET /consumers/risk-queue — backend reads from risk_scores + consumers, returns ranked list.
4. Officer calls POST /cases — backend creates a cases row, writes an audit_log entry (case_assigned).
5. Mobile app (when online) calls GET /cases/mine and caches the result in local SQLite.
6. Inspector fills the inspection form offline; the mobile app writes it to a local, unsynced SQLite row immediately — no network call attempted yet.
7. When connectivity returns, the mobile app's background sync process POSTs to /cases/{id}/inspection; backend writes an inspections row, updates the case's status, uploads the photo to object storage, and writes an audit_log entry (inspection_submitted).
8. Web Dashboard's case list and audit log reflect the update on next fetch — no push mechanism in v1 (per PRD MOB-9/Won't-this-version), so the officer sees it on refresh or next page load.
9. Closed inspection outcomes (confirmed / false_positive) are the labeled data referenced as the model's feedback loop in the ML Strategy document — exported periodically for the next offline retraining cycle, not consumed automatically.

5. Offline-Sync Architecture (Mobile)

Elaborated at the technical level in the Technical Design Document (Section 6); architecturally, the key property is that the mobile app never blocks a user action on network availability. Every write (inspection submission) is local-first; the sync process is a separate, best-effort background concern layered on top, not a precondition for the UI to function.

6. Security Architecture

6.1 Authentication flow

10. Client POSTs credentials to /auth/login.
11. Backend verifies password hash, issues a signed JWT containing user id and role, with an expiry (e.g. 7 days for mobile, given field usage patterns; shorter for web).

12. Client stores the token (secure storage on mobile; httpOnly-cookie or memory on web) and attaches it as a Bearer token on every subsequent request.
13. Backend verifies the JWT signature and expiry on every request; role is read from the token claims, not re-queried from the DB on each call, for efficiency.

6.2 Authorization

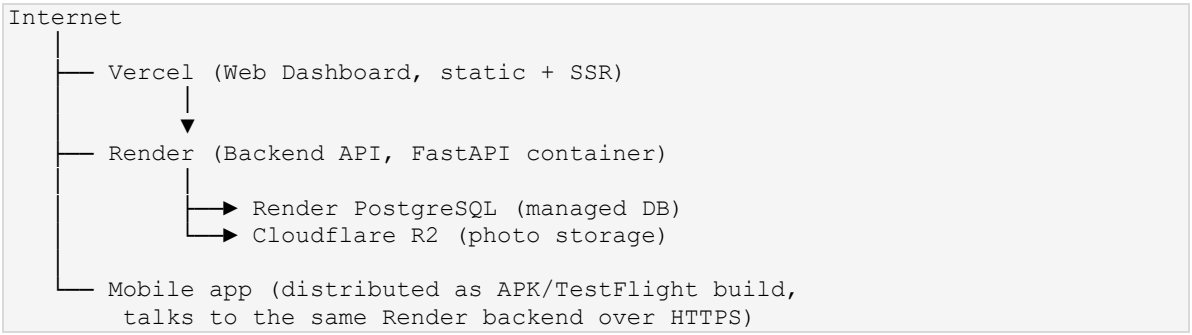
- Officer-only endpoints (case assignment, model performance) reject inspector-role tokens with 403.
- Inspector-only endpoints (/cases/mine, inspection submission) are scoped to the authenticated user's own assigned cases — an inspector cannot submit an inspection for a case assigned to someone else.

6.3 Data protection

- No consumer PII beyond meter/consumer ID and address is stored or displayed — consistent with the PRD's non-functional security requirement.
- Photos and GPS data are only ever linked to a case, never exposed via a public/unauthenticated URL.
- All traffic over HTTPS; no plaintext credentials or tokens in logs.

7. Deployment Topology

Maps directly onto the free-tier hosting plan in the Technical Design Document — restated here as a topology view rather than a hosting-choice table:



8. What This Architecture Deliberately Does Not Include (v1)

- No message queue / event bus — the system's write volume (single-digit consumers/cases per day at demo scale) does not justify one; direct DB writes are sufficient.
- No microservices split — backend is a single deployable service; splitting ML scoring into its own service is a future option if scoring load ever grows, not a v1 concern.
- No automated retraining pipeline — the feedback loop (Section 4, step 9) is architecturally supported (data is captured and exportable) but retraining itself remains a manual, offline step for v1, matching the ML Strategy document's scope.
- No push notifications — would require a persistent connection service (e.g. Firebase Cloud Messaging); explicitly deferred per PRD MOB-9.